

Feasibility Study

Synapse — A Hierarchical Multi-Agent Personal Productivity Assistant

Prepared as a technical, operational, economic, schedule, and legal feasibility assessment

Keywords: Multi-Agent Systems · LLMs · LangGraph · Conversational AI · Telegram Automation · ReAct Agents · Task Orchestration

Table of Contents

Table of Contents.....	2
1. Executive Summary	2
2. Key Features	3
2.1 Conversational Interface (Telegram)	3
2.2 Manager Agent — Intent Understanding & Orchestration	3
2.3 Email Agent (Gmail).....	4
2.4 Calendar Agent (Google Calendar).....	4
2.5 Notion Agent (Tasks)	4
2.6 Slack Agent	4
2.7 Persistent, Per-User Memory	4
2.8 Asynchronous, Non-Blocking Processing	4
2.9 Provider-Agnostic Model Layer	4
2.10 End-to-End Observability	5
3. Technical Feasibility	5
3.1 Architecture Pattern	5
3.2 Integration Feasibility (per service)	5
3.3 ReAct Sub-Agent Reliability	6
3.4 Memory and State.....	6
3.5 Concurrency and Webhook Reliability	6
3.6 Model-Provider Abstraction	6
3.7 Observability	6
4. Operational Feasibility.....	7
5. Economic Feasibility	7
6. Schedule Feasibility	8
7. Legal, Compliance, and Security Feasibility.....	8
7.1 OAuth Scope Verification	8
7.2 Data Handling	9
7.3 Security.....	9
8. Alternatives Considered	9
9. Overall Recommendation	10

1. Executive Summary

Synapse proposes a supervisor–worker multi-agent system, built on LangGraph, that unifies Gmail, Google Calendar, Notion, and Slack behind a single Telegram conversational interface. A Manager agent decomposes and routes user requests to four specialised ReAct sub-agents, backed by SQLite-persisted per-user memory and a provider-agnostic LLM layer (GPT-4o, Gemini, Groq).

This study concludes the project is feasible as a demonstrable prototype or capstone-level system, and feasible with moderate additional engineering as a small-team internal tool. It is **not yet feasible as a multi-tenant commercial product** without material investment in OAuth verification, security hardening, and cost controls. The core technical risk is not the LangGraph orchestration — which is well-trodden —

but the aggregate reliability of four independently fallible integrations chained behind a single LLM-driven router, where errors compound multiplicatively rather than additively.

Each dimension is assessed with a rating: Feasible, Feasible with caveats, Not currently feasible.

Dimension	Rating
Technical	Feasible with caveats
Operational	Feasible
Economic	Feasible (prototype / smallscale)
Schedule	Feasible
Legal / Compliance	Feasible with caveats
Security	Feasible with caveats

2. Key Features

Synapse's value proposition rests on four domain sub-agents operating under a single Manager, plus the cross-cutting infrastructure (memory, concurrency, model layer, observability) that makes the multi-agent behavior reliable and inspectable. Each feature below is what a user actually experiences or what an engineer actually maintains — not an implementation detail.

2.1 Conversational Interface (Telegram)

Users interact entirely through a normal Telegram chat — no separate app, login, or dashboard. Messages arrive via a Telegram Bot API webhook into a FastAPI service, so the “interface” is just the chat window the user already has open.

2.2 Manager Agent — Intent Understanding & Orchestration

The Manager is the single entry point for every user message and is responsible for:

- Interpreting intent from free-form natural language, including compound, multi-service requests in one message.
- Decomposing a compound request into an ordered set of sub-tasks (e.g., check availability, then send an email, then log a task).
- Delegating each sub-task to the correct sub-agent (Email, Calendar, Notion, or Slack).
- Verifying and aggregating sub-agent results into one concise reply, rather than returning four raw tool outputs.

2.3 Email Agent (Gmail)

Handles the email domain end-to-end:

- Reading and summarising messages or threads on request (“what did Priya send me this week?”).
- Composing and sending mail from natural-language instructions.
- Resolving a name to an email address via the Google People API, so users can say “email Priya” instead of typing an address.

2.4 Calendar Agent (Google Calendar)

Handles scheduling:

- Retrieving upcoming events or a day's/week's schedule.
- Checking availability (free/busy) before committing to a time.
- Creating new events directly from a natural-language request.

2.5 Notion Agent (Tasks)

Handles the personal to-do list stored in Notion:

- Retrieving existing tasks, optionally filtered by status or due date.
- Creating new tasks with a status and due date set from the conversation, without the user opening Notion.

2.6 Slack Agent

Handles workspace messaging:

- Reading recent messages from a channel or DM the user asks about.
- Sending messages to a channel or person on the user's behalf.

2.7 Persistent, Per-User Memory

Conversation state is checkpointed to SQLite and keyed by each user's Telegram chat ID, so multi-turn context (“what about tomorrow instead?” referring to a request two messages earlier) survives across sessions, not just within one exchange.

2.8 Asynchronous, Non-Blocking Processing

The FastAPI webhook handles messages asynchronously so it can return quickly to Telegram (avoiding Telegram's retry-on-timeout behavior) while the Manager's multi-agent reasoning continues in the background, keeping the system responsive under concurrent load from multiple users or messages.

2.9 Provider-Agnostic Model Layer

The LLM powering the Manager and sub-agents is swappable across OpenAI GPT-4o, Google Gemini, and Groq-hosted models, so the system isn't locked to a single provider's pricing, latency, or availability.

2.10 End-to-End Observability

Every agent's reasoning trace and tool invocation is captured via LangSmith, giving a full, inspectable record of why the Manager routed a request the way it did and what each sub-agent actually did — essential for debugging multi-agent handoffs, which are otherwise opaque from logs alone.

3. Technical Feasibility

3.1 Architecture Pattern

The supervisor–worker (hierarchical) pattern is a mature, well-documented LangGraph pattern, and is the correct choice here: it isolates tool schemas per domain (Gmail vs. Notion vs. Slack tool signatures don't pollute a single agent's context), allows independent iteration on each sub-agent's prompt/tools, and gives the Manager a narrow, auditable job — classify, decompose, delegate, aggregate. This is sound and low-risk.

Caveat — task decomposition correctness. The Manager's job of decomposing a compound request (“check if I'm free Thursday and if so email Priya and add a Notion task”) into an ordered sub-task graph is itself an unsolved-in-general LLM planning problem. Two failure modes should be designed for:

- Ordering errors — e.g., sending the email before confirming calendar availability.
- Silent partial failure — three of four sub-tasks succeed, one fails, and the Manager's aggregation step must not report overall success. This requires an explicit partial-success reporting contract between sub-agents and Manager.

3.2 Integration Feasibility (per service)

Integration	Feasibility	Key constraint
Telegram Bot API	High	Well-documented, generous free tier, webhook model is simple.
Gmail API	Medium	Requires OAuth 2.0 consent screen; sending mail requires the gmail.send scope, which Google treats as sensitive — see Section 7.1.
Google People API	Medium	Contact resolution (“email it to Priya”) is useful but ambiguousname resolution needs an explicit disambiguation turn, not silent guessing.
Google Calendar API	High	Mature API; the freebusy endpoint directly supports availability checks.
Notion API	High	Straightforward REST API; database schema (status/due-date properties) must be pre-provisioned and mapped, a one-time setup cost per user.
Slack API	Medium	Reading messages requires broader scopes (channels:history, possibly groups:history) that Slack's app-approval process scrutinises more than sending.

None of these integrations are individually hard. The technical risk is compounding: if each of six API calls in a chained workflow has a 97% success rate, a four-step chain succeeds only ~89% of the time. This is the single most important technical risk to design around (timeouts, retries, partial-failure messaging to the user) rather than an architecture-level flaw.

3.3 ReAct Sub-Agent Reliability

ReAct loops per sub-agent are appropriate for read/write tool use with a small, well-scoped toolset (3–6 tools per agent is a reasonable target). Risks to manage:

- Irreversible actions (sending an email, posting to Slack) should not be auto-executed on a single ReAct pass without either a confirmation step for high-stakes actions, or very tight tool descriptions that constrain the model's discretion. Recommend confirmation-before-send as a default.
- Tool-result grounding: sub-agents summarising email/Slack content must not fabricate content when a tool call returns empty or errors — this needs explicit prompt-level guardrails and a post-hoc check in the Manager's verification step.

3.4 Memory and State

SQLite-backed LangGraph checkpoints keyed by Telegram chat ID is a sound, low-complexity choice for single-instance deployment. Caveats:

- SQLite will not scale horizontally — if the FastAPI service ever runs as multiple replicas behind a load balancer, SQLite becomes a bottleneck/corruption risk. Acceptable at prototype/small-team scale; flag as a migration item (e.g., to Postgres) if usage grows.
- No stated memory-pruning strategy — unbounded conversation history per user will eventually inflate token costs and latency. A rolling-window or summarisation strategy should be budgeted in.

3.5 Concurrency and Webhook Reliability

Async/non-blocking FastAPI handling is the right call for Telegram's webhook delivery model, since Telegram expects a fast 200 OK and will retry on timeout, which can otherwise cause duplicate-processing bugs. This is feasible with standard async handlers plus a background task queue so long-running agent reasoning doesn't block the webhook response — a naive synchronous handler will cause Telegram to redeliver messages and trigger duplicate agent runs.

3.6 Model-Provider Abstraction

Supporting GPT-4o, Gemini, and Groq-hosted models behind a common interface is feasible, but introduces real engineering cost: tool-calling behavior, function-schema adherence, and reasoning quality differ meaningfully across providers, especially for the Manager's decomposition step. Recommend treating “provider-agnostic” as an architectural property (the code supports swapping) rather than a claim that quality is provider-invariant.

3.7 Observability

LangSmith tracing for tool invocation and reasoning is a good fit and low-cost to add given the LangGraph foundation. This materially de-risks debugging multi-agent handoff bugs, which are otherwise very hard to diagnose from logs alone.

Overall technical verdict: Feasible with caveats — no component is individually risky, but the system's reliability is bounded by the weakest chained integration, and this needs explicit engineering attention (retries, partial-failure semantics, confirmation gates on irreversible actions).

4. Operational Feasibility

Strengths:

- Telegram as the interface is a strong choice for adoption — no app install, cross-platform, familiar chat UX, and the Bot API is free and low-friction.
- Natural-language access to four services genuinely reduces context-switching for the target user (an individual knowledge worker), which is the project's core value proposition and is directly testable.
- Per-user setup (OAuth grants, Notion integration token, Slack app install) is a real onboarding step, but a one-time cost rather than per-session friction. **Constraints:**
- This is fundamentally a single-user or small-trusted-group tool in its current design. Operationally fine for a personal assistant or team pilot; not yet a multi-tenant SaaS operational model (see Section 7).
- Users must trust the bot with read/write access to email and messaging — a meaningful trust threshold. Rollout should start with read-heavy, low-stakes actions (calendar lookups, task creation) before enabling send-capable actions.

Verdict: Feasible for its evident intended scope (personal productivity tool / small-team pilot / academic demonstration).

5. Economic Feasibility

Rough cost drivers, assuming a single active user with moderate daily usage (~20–40 assistant interactions/day):

Cost item	Estimate	Notes
LLM API calls (Manager + sub-agents)	~\$5–30/month	Multi-agent hops mean each user message can trigger 2–5 LLM calls; cheaper tiers (GPT-4o-mini, Gemini
		Flash, Groq) keep this low.
Hosting (FastAPI service)	~\$0–10/month	A small VM or serverless container is sufficient at prototype scale.
Telegram Bot API	Free	No cost.
Gmail / Calendar / People API	Free	Within Google's standard free quota for personal-scale use.
Notion API	Free	No cost at this scale.
Slack API	Free	No cost at this scale; paid tiers relate to the Slack workspace plan, not API usage.

LangSmith tracing	Free tier likely sufficient	Paid tiers only needed at higher trace volume.
-------------------	-----------------------------	--

Verdict: Economically feasible at prototype/personal/small-team scale — total cost is plausibly under \$50/month. The main scaling cost risk is LLM token spend growing with unbounded conversation history and multi-hop agent chains per request; both are manageable with the mitigations noted in Section 3. Cost at multi-tenant commercial scale is a materially different analysis not covered by this study's assumptions.

6. Schedule Feasibility

A plausible phased build-out for a single developer or small team:

Phase	Scope	Estimate
1. Foundation	Telegram webhook + FastAPI skeleton, single LLM call echo bot, LangSmith wired up	1–2 weeks
2. Single-domain agent	One sub-agent end-to-end (e.g., Calendar: read + freebusy + create event) with ReAct loop and real OAuth	1–2 weeks
3. Manager + routing	Manager decomposition/delegation logic, second and third subagents (Notion, Email)	2–3 weeks
4. Remaining integration + memory	Slack agent, SQLite checkpointing, multi-turn context	1–2 weeks
5. Hardening	Partial-failure handling, confirmation gates on sends, contact disambiguation, retries	2–3 weeks
6. Evaluation	Test scenarios, latency/cost measurement, write-up	1–2 weeks

Total: roughly 8–14 weeks for a functioning, reasonably robust prototype by one to two developers — consistent with a single-semester academic project or a focused individual side project.

Verdict: Feasible within a standard project/thesis timeframe.

7. Legal, Compliance, and Security Feasibility

This is the dimension most likely to be underestimated, and deserves explicit attention rather than being folded into “technical.”

7.1 OAuth Scope Verification

- Gmail's gmail.send and broader read scopes fall under Google's restricted-scopes policy, requiring a security assessment (CASA) for apps requesting them beyond a 100-user testing cap, or for public availability. For a personal/small-team deployment kept under Google's unverified-app testing limits, this is a non-issue; for wider release, it's a real compliance gate with cost and lead time attached.
- Slack's app approval for workspace-installed bots with message-history scopes is comparatively lighter, but still requires the installing workspace admin's consent and review of requested scopes.

7.2 Data Handling

- The system will, by necessity, have transient access to email content, calendar details, and Slack messages — some of which may be sensitive. Passing this content through third-party LLM APIs (OpenAI, Google, Groq) means it leaves the user's own infrastructure, and should be clearly disclosed to any user of the system.
- If deployed for anyone other than the developer themselves, this raises real questions under frameworks like GDPR — specifically data minimisation, lawful basis for processing, and the fact that LLM providers act as sub-processors. Manageable for personal use; a genuine blocker for redistribution without a privacy policy and consent flow.

7.3 Security

- Credential storage: OAuth refresh tokens and API tokens are high-value secrets and need encryption at rest, not plain SQLite rows, if this moves beyond a local prototype.
- Chat-ID authentication: using Telegram chat ID as the sole key for per-user memory is reasonable given Telegram's own authentication of the chat, but should not be assumed sufficient authorization for binding a specific set of Google/Slack/Notion credentials without a deliberate account-linking step during onboarding.
- Prompt injection via fetched content: an email body or Slack message the agent reads could contain adversarial text designed to manipulate the agent. Sub-agents that read external content and also have write/send capability are the highest-risk combination in this architecture and warrant the same confirmation-gate caution as Section 3.3.

Verdict: Feasible for personal/self-use and small trusted-team deployment. Not yet feasible for distribution to arbitrary third-party users without additional work on scope verification, credential security, and a privacy/consent framework.

8. Alternatives Considered

- Single monolithic agent with all tools attached: simpler initially, but tool-schema bloat degrades routing accuracy as the toolset grows — the hierarchical design is the right tradeoff for four distinct service domains.
- Zapier/Make-style deterministic automation instead of LLM-driven agents: more reliable and cheaper per-action, but loses the natural-language, compound-request flexibility that is the actual point of Synapse — a known tradeoff, not an unqualified strength.
- Slack or Discord instead of Telegram: viable alternatives, but Telegram's Bot API is simpler to stand up and doesn't require workspace admin approval to test, supporting the schedule feasibility in Section 6.

9. Overall Recommendation

Proceed with the project as scoped for a prototype / personal productivity tool / academic demonstration. The architecture is sound and the timeline is realistic. Prioritise, in this order:

1. Confirmation gates on all irreversible send/write actions (email send, Slack post) before those tools are exposed to autonomous ReAct execution.
2. Explicit partial-failure reporting contract between sub-agents and the Manager, so aggregated replies never overstate success.
3. Prompt-injection awareness for any sub-agent that both reads external content and holds write capability.
4. Treat OAuth scope verification and credential encryption as a distinct, non-trivial task if the system is ever shared beyond its original developer/user.

None of these are reasons to reconsider the project — they are the specific, addressable risks that separate a working demo from a trustworthy assistant, and are worth budgeting time for from the outset rather than retrofitting later.